

LAB REPORT

Clocks and Timers

3220102382 潘谷雨

2024.12.26

1. Problem Description

(1) 3-digit BCD counter

Implement a 3-digit BCD counter. Display the contents of the counter on the 7-segment displays, HEX2–0. Derive a control signal, from the 50-MHz clock signal provided on the Altera DE2 board, to increment the contents of the counter at one-second intervals. Use the pushbutton switch KEY0 to reset the counter to 0.

1. Create a new Quartus II project which will be used to implement the desired circuit on the DE2 board.
2. Write a VHDL file that specifies the desired circuit.
3. Include the VHDL file in your project and compile the circuit.
4. Simulate the designed circuit to verify its functionality.
5. Assign the pins on the FPGA to connect to the 7-segment displays and the pushbutton switch, as indicated in the User Manual for the DE2 board.
6. Recompile the circuit and download it into the FPGA chip.
7. Verify that your circuit works correctly by observing the display.

(2) time-of-day clock

Design and implement a circuit on the DE2 board that acts as a time-of-day clock. It should display the hour (from 0 to 23) on the 7-segment displays HEX7–6, the minute (from 0 to 60) on HEX5–4 and the second (from 0 to 60) on HEX3–2. Use the switches SW15–0 to preset the hour and minute parts of the time displayed by the clock.

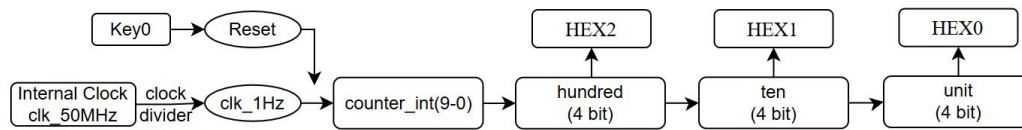
(3) reaction timer

Design and implement on the DE2 board a reaction-timer circuit. The circuit is to operate as follows:

1. The circuit is reset by pressing the pushbutton switch KEY0.
2. After an elapsed time, the red light labeled LEDR0 turns on and a four-digit BCD counter starts counting in intervals of milliseconds. The amount of time in seconds from when the circuit is reset until LEDR0 is turned on is set by switches SW7–0.
3. A person whose reflexes are being tested must press the pushbutton KEY3 as quickly as possible to turn the LED off and freeze the counter in its present state. The count which shows the reaction time will be displayed on the 7-segment displays HEX2–0.

2. Design Formulation

(1) 3-digit BCD counter

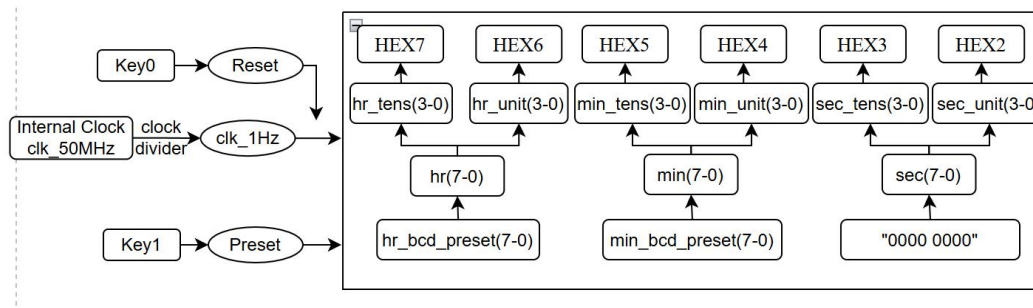


Pic.1.1 design formulation of 3-digit BCD counter

The 50MHz high frequency internal clock signal is passed through a clock divider to generate a 1Hz clock signal. A 10-bit counter is used, driven by the 1Hz clock signal. The reset logic is controlled by the Key0 switch. When SW0 is activated, the counter is reset to zero.

The output of the counter is a 10-bit binary number that is split into three 4-bit parts: unit, ten, and hundred, and is displayed on the corresponding displays (HEX0, HEX1, HEX2) by means of a seven-segment code.

(2) time-of-day clock

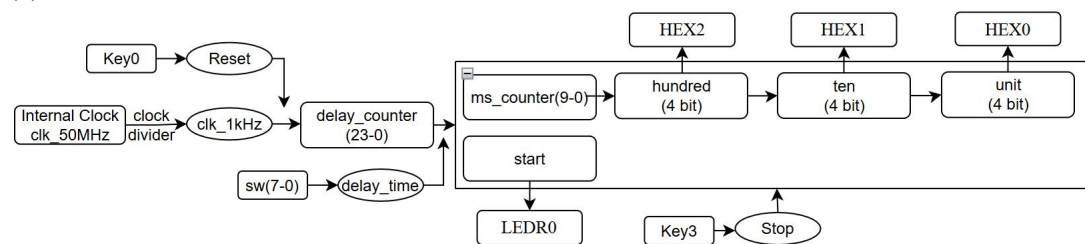


Pic.2.1 design formulation of time-of-day clock

By means of a clock divider, the 50 MHz clock signal is divided into 1Hz clock signals, which are used as the clock source for the counter, ensuring that the counter is incremented once per second. The system contains three main counters: hours (hr), minutes (min), and seconds (sec), each of which is 8-bits wide, and the output is divided into two 4-bit sections representing the tens and the ones, with each 4-bit output being displayed via a single display (HEX0-HEX7). When the seconds counter reaches 59, it generates a rounding signal to increment the minutes counter by 1. Similarly, when the minutes counter reaches 59, it generates a rounding signal to increment the hours counter by 1.

The system has two reset switches: Key0 is used to reset the whole system and Key1 is used to preset the time. The system allows presetting of hour and minute values via hr_bcd_preset and min_bcd_preset. These presets are loaded in BCD format.

(3) reaction timer



Pic.3.1 design formulation of reaction timer

The 50MHz clock signal is reduced in frequency by a clock divider (clock divider) to generate a 1kHz clock signal (clk_1kHz). The reset signal is controlled via key Key0 and is used to initialize or reset the entire system. Use the switches (sw[7:0]) to set the delay time (delay_time), a value that determines how long the timer has to wait before it starts timing. The delay counter (delay_counter[23:0]) is used to count the delay time. When the counter reaches the value of the delay time, the timer starts timing.

The millisecond counter (ms_counter[9:0]) is used to count the number of milliseconds elapsed and is driven by a 1kHz clock signal. The output is divided into three parts: hundreds (hundred), tens (ten), and units (unit), each of which is 4-bits wide and is displayed by three digital tubes (HEX0, HEX1, HEX2). The stop signal is controlled by the key Key3, which is used to stop the timer, and the start status of the timer is indicated by the LEDR0.

3. Design Entry

(1) 3-digit BCD counter

binary_to_bcd.vhd:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity binary_to_bcd is
6  Port (
7      V : in STD_LOGIC_VECTOR (9 downto 0); -- 10-bit binary input
8      H : out STD_LOGIC_VECTOR (3 downto 0); -- Hundreds digit output
9      T : out STD_LOGIC_VECTOR (3 downto 0); -- Tens digit output
10     U : out STD_LOGIC_VECTOR (3 downto 0) -- Units digit output
11 );
12 end binary_to_bcd;
13
14 architecture Behavioral of binary_to_bcd is
15     signal V_int : unsigned(9 downto 0); -- 输入的整数值
16
17 begin
18     V_int <= unsigned(V); -- 将输入转换为无符号整数
19
20     process(V_int)
21     variable hundred, ten, unit : unsigned(3 downto 0) := (others => '0');
22     variable temp : unsigned(9 downto 0) := (others => '0');
23     begin
24         temp := V_int;
25
26         -- 计算百位
27         if temp >= 100 then
28             hundred := resize(temp / 100, hundred'length);
29             temp := temp mod 100;
30         else
31             hundred := "0000";
32         end if;
33
34         -- 计算十位
35         if temp >= 10 then
36             ten := resize(temp / 10, ten'length);
37             temp := temp mod 10;
38         else
39             ten := "0000";
40         end if;
41
42         -- 剩余的是个位
43         unit := resize(temp, unit'length);

```

```

45         -- 输出BCD
46         H <= std_logic_vector(hundred);
47         T <= std_logic_vector(ten);
48         U <= std_logic_vector(unit);
49     end process;
50
51 end Behavioral;

```

seven_segment_decoder.vhd:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity seven_segment_decoder is
6  Port ( bin : in  STD_LOGIC_VECTOR(3 downto 0);
7        seg : out STD_LOGIC_VECTOR(6 downto 0));
8  end seven_segment_decoder;
9
10 architecture Behavioral of seven_segment_decoder is
11 begin
12     process(bin)
13     begin
14         case bin is
15             when "0000" => seg <= "1000000"; -- 0
16             when "0001" => seg <= "1111001"; -- 1
17             when "0010" => seg <= "0100100"; -- 2
18             when "0011" => seg <= "0110000"; -- 3
19             when "0100" => seg <= "0011001"; -- 4
20             when "0101" => seg <= "0010010"; -- 5
21             when "0110" => seg <= "0000010"; -- 6
22             when "0111" => seg <= "1111000"; -- 7
23             when "1000" => seg <= "0000000"; -- 8
24             when "1001" => seg <= "0010000"; -- 9
25             when "1010" => seg <= "0001000"; -- A
26             when "1011" => seg <= "0000011"; -- B
27             when "1100" => seg <= "1000110"; -- C
28             when "1101" => seg <= "0100001"; -- D
29             when "1110" => seg <= "0000110"; -- E
30             when others => seg <= "0001110"; -- F
31         end case;
32     end process;
33 end Behavioral;

```

clock_divider.vhd:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity clock_divider is
6  Port ( clk_50MHz : in  STD_LOGIC;
7        reset      : in  STD_LOGIC;
8        clk_1Hz    : out STD_LOGIC);
9  end clock_divider;
10
11 architecture Behavioral of clock_divider is
12     signal count : unsigned(25 downto 0) := (others => '0');
13 begin
14     process(clk_50MHz, reset)
15     begin
16         if reset = '0' then
17             count <= (others => '0');
18             clk_1Hz <= '0';
19         elsif rising_edge(clk_50MHz) then
20             if count = 100 - 1 then--if count = 50000000 - 1 then--
21                 count <= (others => '0');
22                 clk_1Hz <= '1';
23             else
24                 count <= count + 1;
25                 if count = 50 - 1 then--if count = 25000000 - 1 then--
26                     clk_1Hz <= '0';
27                 end if;
28             end if;
29         end if;
30     end process;
31 end Behavioral;

```


three_digit_bcd_counter:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity three_digit_bcd_counter is
6  Port (
7      clk_50MHz : in STD_LOGIC;    -- 50 MHz clock input
8      reset : in STD_LOGIC;        -- Reset signal from KEY0
9      hex0, hex1, hex2 : out STD_LOGIC_VECTOR(6 downto 0);    -- 7-segment displays
10     counter : buffer unsigned(9 downto 0) := (others => '0'); -- 10-bit counter for BCD (0 to 999)
11     clk_1Hz : buffer STD_LOGIC;
12     hundred, ten, unit : buffer STD_LOGIC_VECTOR (3 downto 0) -- Digits for display
13 );
14 end three digit bcd counter;
15
16 architecture Behavioral of three_digit_bcd_counter is
17
18     component binary_to_bcd is
19     Port (
20         V : in STD_LOGIC_VECTOR (9 downto 0);
21         H : out STD_LOGIC_VECTOR (3 downto 0);
22         T : out STD_LOGIC_VECTOR (3 downto 0);
23         U : out STD_LOGIC_VECTOR (3 downto 0)
24     );
25     end component;
26
27     component seven_segment_decoder is
28     Port (
29         bin : in STD_LOGIC_VECTOR(3 downto 0);
30         seg : out STD_LOGIC_VECTOR(6 downto 0)
31     );
32     end component;
33
34     component clock_divider is
35     Port ( clk_50MHz : in  STD_LOGIC;
36           reset : in  STD_LOGIC;
37           clk_1Hz : out  STD_LOGIC);
38     end component;
39
40     begin
41         -- Clock Divider to generate a 1Hz clock from the 50MHz clock
42         U_clock_divider: clock_divider port map (clk_50MHz, reset, clk_1Hz);
43
44         -- BCD Counter
45         process(clk_1Hz, reset)
46         begin
47             if reset = '0' then
48                 counter <= (others => '0');
49             elsif rising_edge(clk_1Hz) then
50                 if counter = 999 then
51                     counter <= (others => '0');
52                 else
53                     counter <= counter + 1;
54                 end if;
55             end if;
56         end process;
57
58         -- Convert 10-bit binary to BCD
59         U_binary_to_bcd: binary_to_bcd port map (std_logic_vector(counter), hundred, ten, unit);
60
61         -- Convert BCD to 7-segment display code
62         U_seven_segment_decoder_hundreds: seven_segment_decoder port map (hundred, hex2);
63         U_seven_segment_decoder_tens: seven_segment_decoder port map (ten, hex1);
64         U_seven_segment_decoder_units: seven_segment_decoder port map (unit, hex0);
65
66     end Behavioral;
```

(2)time-of-day clock

seven_segment_decoder.vhd and clock_divider.vhd are files included in part (1).

binary_to_bcd.vhd:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity binary_to_bcd is
6  Port (
7      V : in STD_LOGIC_VECTOR (7 downto 0); -- 10-bit binary input
8      T : out STD_LOGIC_VECTOR (3 downto 0); -- Tens digit output
9      U : out STD_LOGIC_VECTOR (3 downto 0) -- Units digit output
10 );
11 end binary_to_bcd;
12
13 architecture Behavioral of binary_to_bcd is
14     signal V_int : unsigned(7 downto 0); -- 输入的整数值
15
16 begin
17     V_int <= unsigned(V); -- 将输入转换为无符号整数
18
19     process(V_int)
20         variable ten, unit : unsigned(3 downto 0) := (others => '0');
21         variable temp : unsigned(7 downto 0) := (others => '0');
22     begin
23         temp := V_int;
24
25         -- 计算十位
26         if temp >= 10 then
27             ten := resize(temp / 10, ten'length);
28             temp := temp mod 10;
29         else
30             ten := "0000";
31         end if;
32
33         -- 剩余个位
34         unit := resize(temp, unit'length);
35
36         -- 输出BCD
37         T <= std_logic_vector(ten);
38         U <= std_logic_vector(unit);
39     end process;
40
41 end Behavioral;
```

time_of_day_clock:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity time_of_day_clock is
6  Port (
7      clk_50MHz : in STD_LOGIC; -- 50MHz clock input
8      reset : in STD_LOGIC; -- Reset signal
9      preset : in STD_LOGIC; -- Preset signal
10     SW : in STD_LOGIC_VECTOR(15 downto 0); -- Switch inputs for presetting time
11     HEX7, HEX6, HEX5, HEX4, HEX3, HEX2 : out STD_LOGIC_VECTOR(6 downto 0); -- 7-segment
12     clk_1Hz : buffer STD_LOGIC;
13     sec, min, hr : buffer unsigned(7 downto 0) := (others => '0')
14 );
15 end time_of_day_clock;
```

```

17 architecture Behavioral of time_of_day_clock is
18     component clock_divider is
19     port (
20         clk_50MHz : in STD_LOGIC;
21         reset : in STD_LOGIC;
22         clk_1Hz : out STD_LOGIC
23     );
24 end component;
25
26 component seven_segment_decoder is
27 port (
28     bin : in STD_LOGIC_VECTOR(3 downto 0);
29     seg : out STD_LOGIC_VECTOR(6 downto 0)
30 );
31 end component;
32
33 component binary_to_bcd is
34 port (
35     V : in STD_LOGIC_VECTOR (7 downto 0); -- 6-bit binary input
36     T : out STD_LOGIC_VECTOR (3 downto 0); -- Tens digit output
37     U : out STD_LOGIC_VECTOR (3 downto 0) -- Units digit output
38 );
39 end component;
40
41 signal hr_tens, hr_units, min_tens, min_units, sec_tens, sec_units : STD_LOGIC_VECTOR(3 downto 0);
42 signal hr_bcd_preset, min_bcd_preset: STD_LOGIC_VECTOR (7 downto 0);
43 begin
44
45     -- Instantiate the clock divider
46     U_clock_divider: clock_divider port map (clk_50MHz,reset,clk_1Hz);
47
48     -- preset
49     process(preset)
50     begin
51         --hour
52         if unsigned(SW(15 downto 12)) > 2 then
53             hr_bcd_preset(7 downto 4) <= "0010";
54         else
55             hr_bcd_preset(7 downto 4) <= SW(15 downto 12);
56         end if;
57
58         if hr_bcd_preset(7 downto 4) = "0010" and unsigned(SW(11 downto 8)) > 3 then
59             hr_bcd_preset(3 downto 0) <= "0011";
60         elsif unsigned(SW(11 downto 8)) > 9 then
61             hr_bcd_preset(3 downto 0) <= "1001";
62         else
63             hr_bcd_preset(3 downto 0) <= SW(11 downto 8);
64         end if;
65
66         --min
67         if unsigned(SW(7 downto 4)) > 5 then
68             min_bcd_preset(7 downto 4) <= "0101";
69         else
70             min_bcd_preset(7 downto 4) <= SW(7 downto 4);
71         end if;
72
73         if unsigned(SW(3 downto 0)) > 9 then
74             min_bcd_preset(3 downto 0) <= "1001";
75         else
76             min_bcd_preset(3 downto 0) <= SW(3 downto 0);
77         end if;
78     end process;

```



```

80      -- Time counter process
81      process(clk_1Hz, reset, preset)
82      begin
83          if reset = '0' then
84              sec <= (others => '0');
85              min <= (others => '0');
86              hr <= (others => '0');
87          elsif preset = '0' then
88              hr <= unsigned(hr_bcd_preset(7 downto 4))*10 + unsigned(hr_bcd_preset(3 downto 0));
89              min <= unsigned(min_bcd_preset(7 downto 4))*10 + unsigned(min_bcd_preset(3 downto 0));
90              sec <= (others => '0');
91          elsif rising_edge(clk_1Hz) then
92              if sec = 59 then
93                  sec <= (others => '0');
94                  if min = 59 then
95                      min <= (others => '0');
96                      if hr = 23 then
97                          hr <= (others => '0');
98                      else
99                          hr <= hr + 1;
100                      end if;
101                  else
102                      min <= min + 1;
103                  end if;
104              else
105                  sec <= sec + 1;
106              end if;
107          end if;
108      end process;

110      -- Decompose time into tens and units
111      U_hr_bcd: binary_to_bcd port map (std_logic_vector(hr), hr_tens, hr_units);
112      U_min_bcd: binary_to_bcd port map (std_logic_vector(min), min_tens, min_units);
113      U_sec_bcd: binary_to_bcd port map (std_logic_vector(sec), sec_tens, sec_units);
114
115      -- Instantiate the seven segment decoders
116      U_hr_tens: seven_segment_decoder port map (hr_tens, HEX7);
117      U_hr_units: seven_segment_decoder port map (hr_units, HEX6);
118      U_min_tens: seven_segment_decoder port map (min_tens, HEX5);
119      U_min_units: seven_segment_decoder port map (min_units, HEX4);
120      U_sec_tens: seven_segment_decoder port map (sec_tens, HEX3);
121      U_sec_units: seven_segment_decoder port map (sec_units, HEX2);
122
123  end Behavioral;

```

(3)reaction timer

seven_segment_decoder.vhd and binaru_to_bcd.vhd are files included in part (1).

clock_divider.vhd:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity clock_divider is
6      Port ( clk_50MHz : in  STD_LOGIC;
7            reset      : in  STD_LOGIC;
8            clk_1kHz   : out STD_LOGIC);
9  end clock_divider;
10
11 architecture Behavioral of clock_divider is
12     signal count : unsigned(25 downto 0) := (others => '0');
13 begin
14     process(clk_50MHz, reset)
15     begin
16         if reset = '0' then
17             count <= (others => '0');
18             clk_1kHz <= '0';

```



```

19     elsif rising_edge(clk_50MHz) then
20         if count = 10 - 1 then--if count = 50000 - 1 then--
21             count <= (others => '0');
22             clk_1kHz <= '1';
23         else
24             count <= count + 1;
25             if count = 5 - 1 then--if count = 25000 - 1 then--
26                 clk_1kHz <= '0';
27             end if;
28         end if;
29     end if;
30 end process;
31 end Behavioral;

```

reaction_timer.vhd:

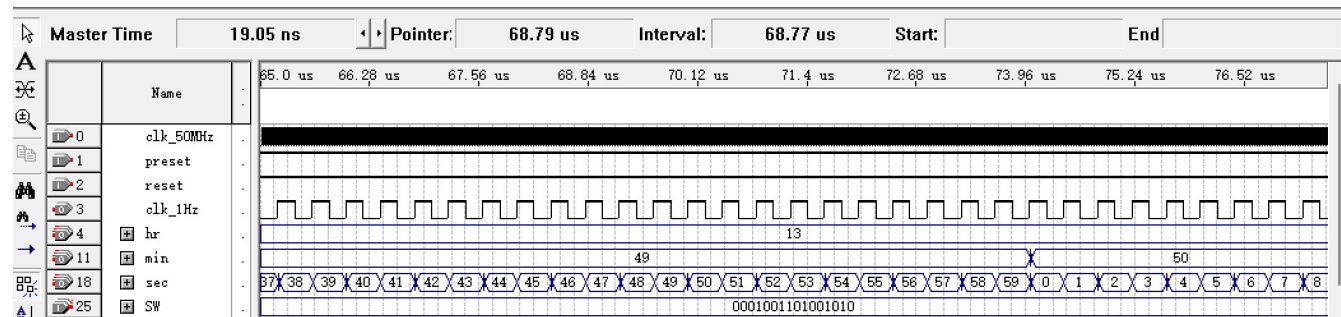
```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity reaction_timer is
6  Port (
7      clk_50MHz : in STD_LOGIC; -- 50MHz clock input
8      reset : in STD_LOGIC; -- Reset button KEY0
9      start : out STD_LOGIC; -- LEDR0 output
10     stop : in STD_LOGIC; -- Stop button KEY3
11     sw : in STD_LOGIC_VECTOR(7 downto 0); -- Switches for delay setting
12     hex2, hex1, hex0 : out STD_LOGIC_VECTOR(6 downto 0);
13     clk_1kHz : buffer STD_LOGIC;
14     delay_counter : buffer unsigned(23 downto 0) := (others => '0');
15     hundred, ten, unit : buffer STD_LOGIC_VECTOR(3 downto 0);
16     ms_counter : buffer unsigned(9 downto 0) := (others => '0');
17     delay_time : buffer unsigned(23 downto 0);
18     start_signal : buffer STD_LOGIC := '0';
19     frozen : buffer STD_LOGIC := '0'
20 );
21 end reaction_timer;
22
23 architecture Behavioral of reaction_timer is
24     component clock_divider is
25     Port ( clk_50MHz : in STD_LOGIC;
26           reset : in STD_LOGIC;
27           clk_1kHz : out STD_LOGIC);
28     end component;
29
30     component seven_segment_decoder is
31     Port (
32         bin : in STD_LOGIC_VECTOR(3 downto 0);
33         seg : out STD_LOGIC_VECTOR(6 downto 0)
34     );
35     end component;
36
37     component binary_to_bcd is
38     Port (
39         V : in STD_LOGIC_VECTOR (9 downto 0);
40         H : out STD_LOGIC_VECTOR (3 downto 0);
41         T : out STD_LOGIC_VECTOR (3 downto 0);
42         U : out STD_LOGIC_VECTOR (3 downto 0)
43     );
44     end component;
45
46 begin
47     -- Instantiate the clock divider to get a 1kHz clock from 50MHz
48     U_clock_divider: clock_divider port map (clk_50MHz, reset, clk_1kHz);
49
50     -- Calculate delay time based on switch settings
51     process(sw)
52         variable delay_temp : unsigned(35 downto 0);
53     begin
54         delay_temp := resize(unsigned(sw), delay_time'length) * to_unsigned(1000, 12);
55         delay_time <= resize(delay_temp, delay_time'length);
56     end process;

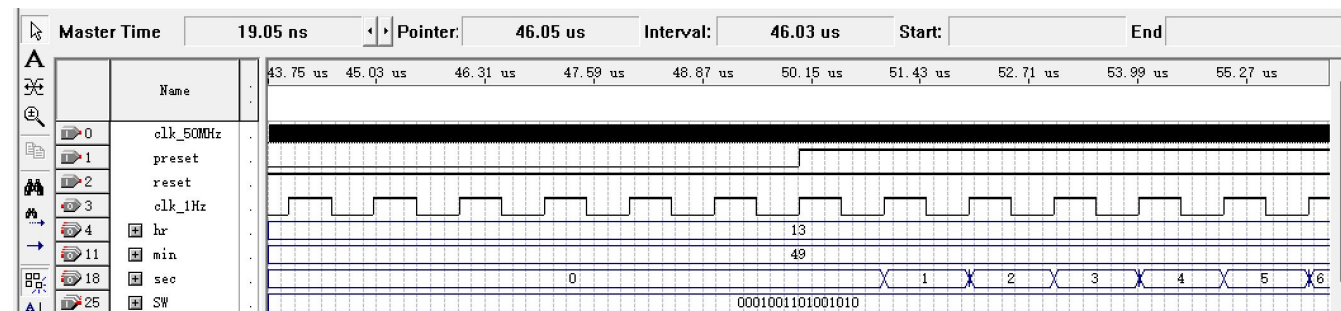
```


(2)time-of-day clock

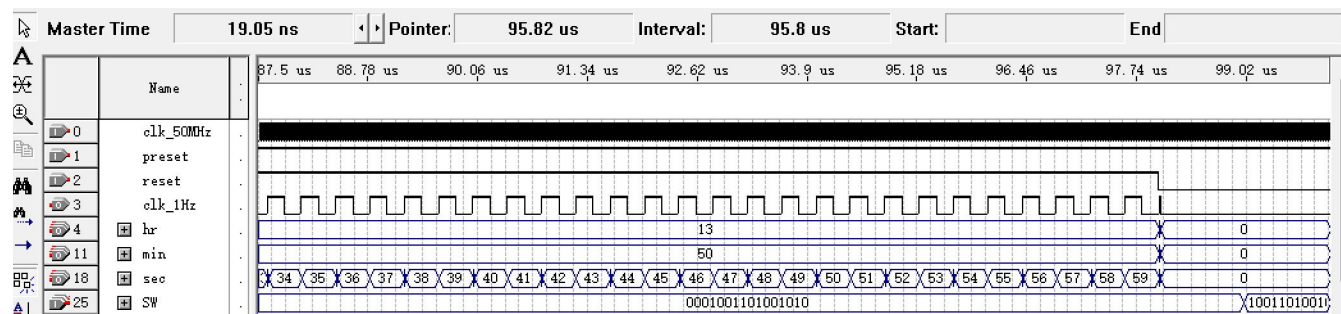
Clock on effect:



Preset on effect

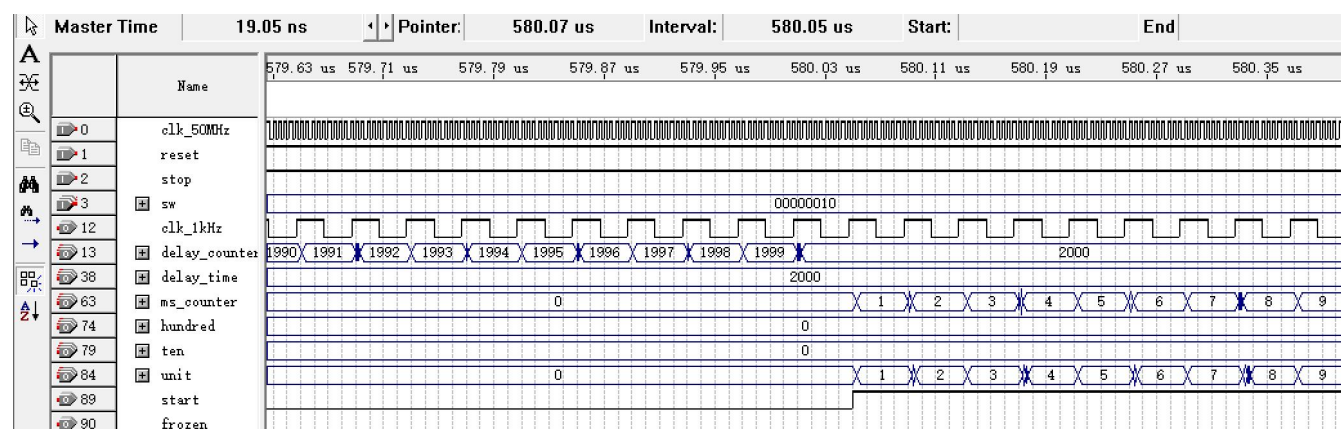


Reset on effect:



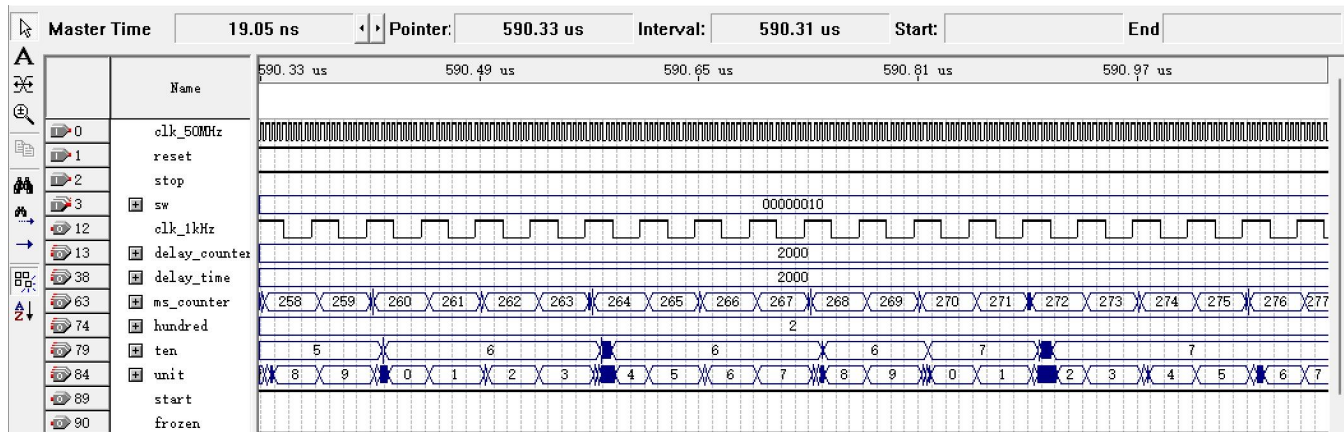
(3)reaction timer

delay_counter on effect:

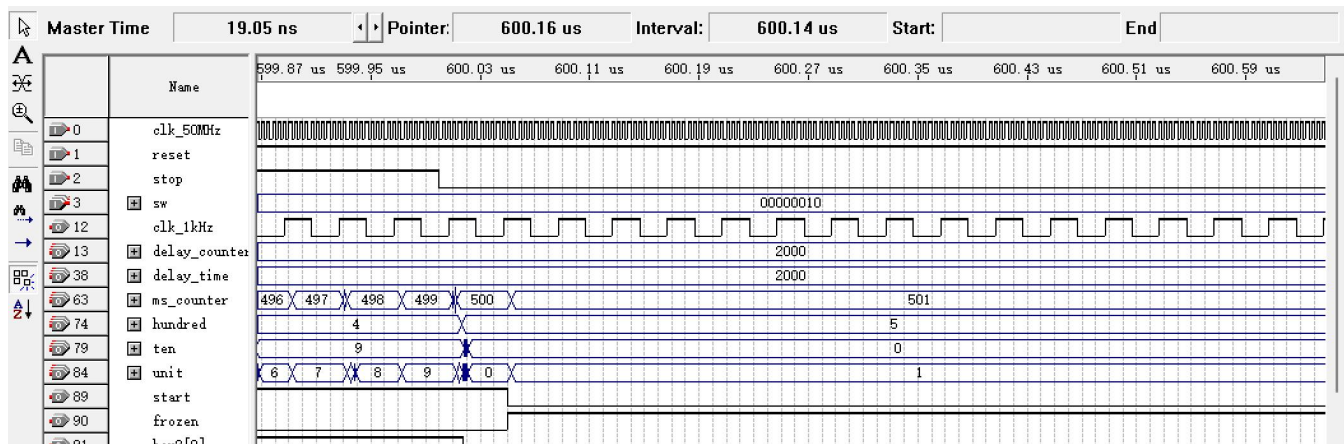


Start

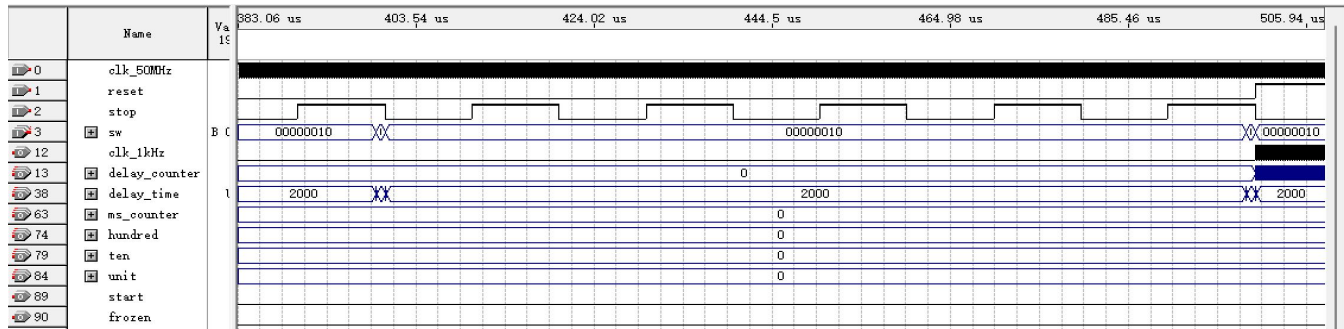
counting:



Stop:



Reset on effect:

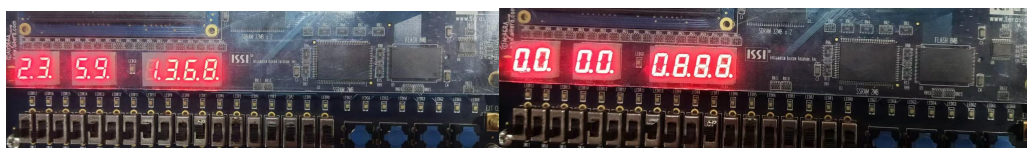


5. Experimental Results

Part (2) and (3) are proved in physical experiment.

(2) time-of-day clock

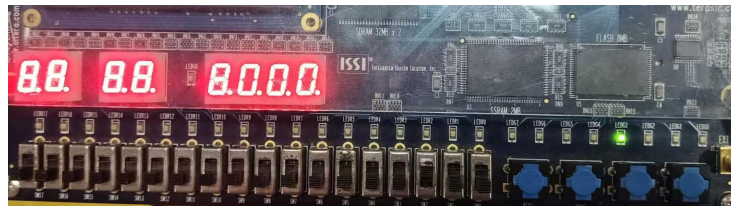
Press Key1 to preset the timer as “23:59:00” and then release Key1 and enable the counter to increase by 1 second. After 1 minute, the timer demonstrates “00:00:08” on the digital tube as Pic.2.2.



Pic.2.2

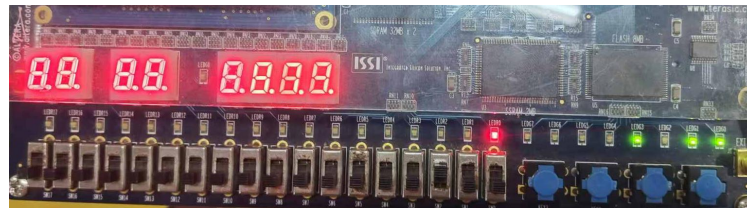
(3) reaction timer

Set delay time as 4000ms. Press Key0 to reset the counter and then release. In Pic.3.2, we can see that within delay time, start light is still off and counter remains zero.



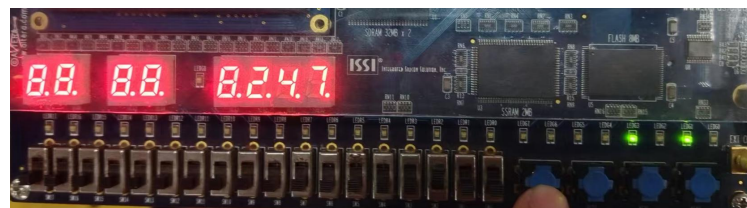
Pic.3.2

After 4000ms, start light turns on and counter starts increasing by 1ms in Pic.3.3.



Pic.3.3

Press Key3 to stop the counter. In Pic.3.4, we can see the counter stops at 247 and start light turns off.



Pic.3.4

6. Discussion and Conclusion

The experiment guided me to gradually master the whole process from theory to practice: first of all, by learning the basic concepts and working principles of digital circuits, I laid a solid theoretical foundation for the subsequent practical operation, and I put this theoretical knowledge into practice in the process of realizing counters and clocks on the actual DE2 board. Meanwhile, the process of writing VHDL code and performing hardware description allowed me to familiarize myself with the syntax and logical structure of a hardware description language, while verifying the design through simulation and synthesis results ensured that my design could operate correctly on actual hardware.

In encountering problems and errors, such as solving VHDL expression errors or optimizing delay time calculations, I gradually developed the ability to solve complex problems. This not only improved my understanding of the interaction between hardware and software, but also increased my confidence in facing real-world engineering problems. Most importantly, I realized that accurate time control is one of the core requirements for many applications, especially in designs involving user interaction, where accurate time response can significantly enhance user experience and system reliability.